

Unit 5

MULTIPROCESSORS :

Characteristics of multiprocessors,
Interconnection structures,
inter processor arbitration,
Inter processor communication and synchronization,
cache coherence,
shared memory multiprocessors.

What is Multiprocessor

& Characteristics of multiprocessors

- A Multiprocessor is a computer system with two or more central processing units (CPUs) share full access to a common RAM.
- The main objective of using a multiprocessor is to boost the system's execution speed, with other objectives being fault tolerance and application matching.
- There are two types of multiprocessors, one is called shared memory multiprocessor and another is distributed memory multiprocessor.
- In shared memory multiprocessors, all the CPUs shares the common memory but
- in a distributed memory multiprocessor, every CPU has its own private memory.

Key Characteristics of Multiprocessor Systems:

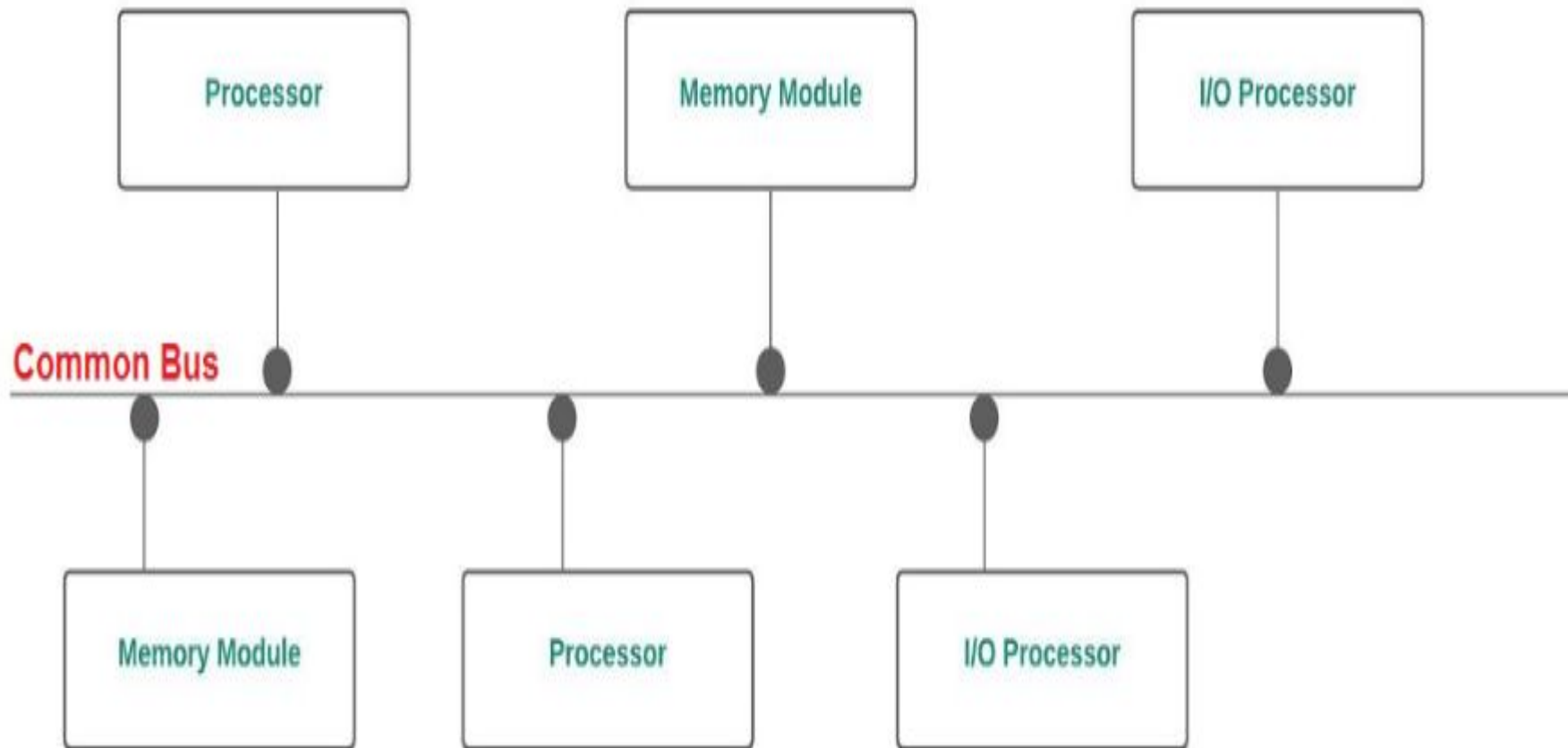
- Parallel Processing: The system can execute multiple instructions or processes at the same time using different processors, significantly improving performance.
- Shared Memory (Tightly Coupled): Processors share a common, global memory space and communicate by reading and writing to this shared area, often coordinated via a system bus.
- Graceful Degradation (Reliability): The system remains operational even if one processor fails. The remaining processors take over, allowing the system to work at a reduced performance rather than failing entirely.
- Resource Sharing: Processors share main memory, I/O devices, and system buses, which increases efficiency and reduces costs compared to independent systems.
- Interconnection Networks: Processors are connected via techniques like time-shared buses, crossbar switches, or multi-port memory to manage communication and data synchronization.
- Cache Coherence: Because each processor may have its own local cache, the system must ensure that all caches maintain the same data consistency, updating shared memory accordingly.
- Scalability: Multiprocessor systems allow for adding more processors to enhance processing power as workload increases.

Interconnection Structure:

The processors must be able to share a set of main memory modules & I/O devices in a multiprocessor system. This sharing capability can be provided through interconnection structures. The interconnection structure that are commonly used can be given as follows -

- Time-shared / Common Bus
- [Cross bar Switch](#)
- [Multiport Memory](#)
- Multistage Switching Network (Covered in 2nd part)
- [Hypercube System](#)

- **Time-Shared Common Bus:** A single path connects all processors and memory; simple but creates a bottleneck.



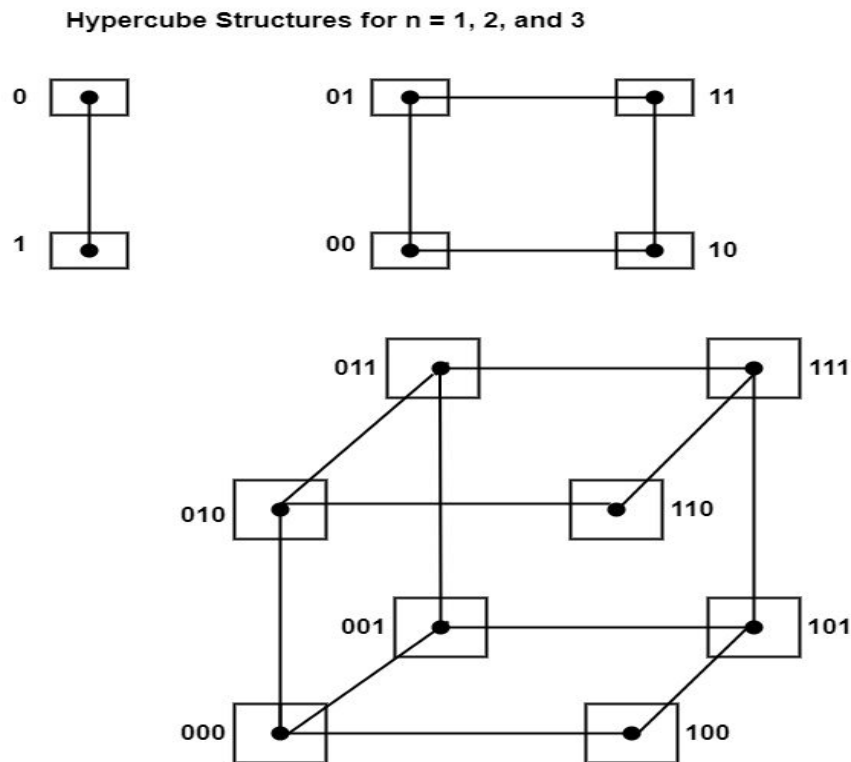
- **Multiport Memory**: Dedicated, direct paths connect each CPU to memory ports, offering high performance but complex hardware.
- **Crossbar Switch**: Enables simultaneous, direct connections between multiple processors and memory modules, providing high throughput.
- **Multistage Switching Network**: Uses interconnected stages of switches to create paths between processors and memory, offering a balance between cost and performance.

Hypercube Interconnection :

The hypercube interconnection is also defined as a binary n-cube multiprocessor. The hypercube is treated to be a loosely coupled system. This system is composed of $N = 2^n$ processors that are linked in an n-dimensional binary cube. Each processor denotes a node of the cube.

- Each processor includes a direct communication route to n other neighbour processors. These routes are associated with the edges of the cube.

The figure displayed the hypercube structure for n, wherein n = 1, 2, and 3.



A one-cube structure includes $n = 1$ and $2^n = 2$. It has two processors that are connected by an individual route. A two-cube structure includes $n = 2$ and $2^n = 4$. It has four nodes that are linked as a square. There are eight nodes associated as a cube in a three-cube structure.

Interprocessor Arbitration

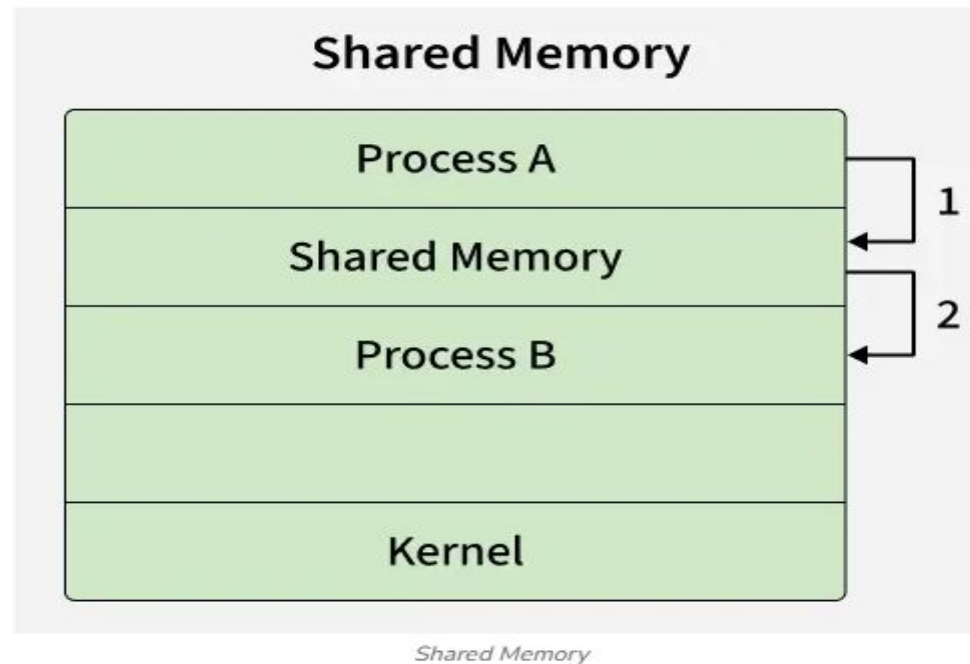
- Arbitration resolves conflicts when multiple processors attempt to access shared resources (e.g., bus, memory).
- Daisy Chaining: Priority is established based on the physical position of the processor on the bus.
- Polling: A controller polls devices to decide which gets access.
- Independent Requesting: Each processor has a separate request and grant line.

Inter processor communication

- Inter-Process Communication or IPC is a mechanism that allows processes to communicate and share data with each other while they are running. Since each process has its own memory space, IPC provides controlled methods for exchanging information and coordinating actions. It helps processes work together efficiently and safely in an operating system.
- It helps processes synchronize their activities, share information and avoid conflicts while accessing shared resources.
- There are two method of IPC, **shared memory and message passing**. An operating system can implement both methods of communication.
- Example: A simple example of IPC is a bank ATM system, where one process reads the card and PIN, another checks the account balance, and a third dispenses cash. These processes communicate and coordinate to complete the transaction correctly.

1. Shared Memory

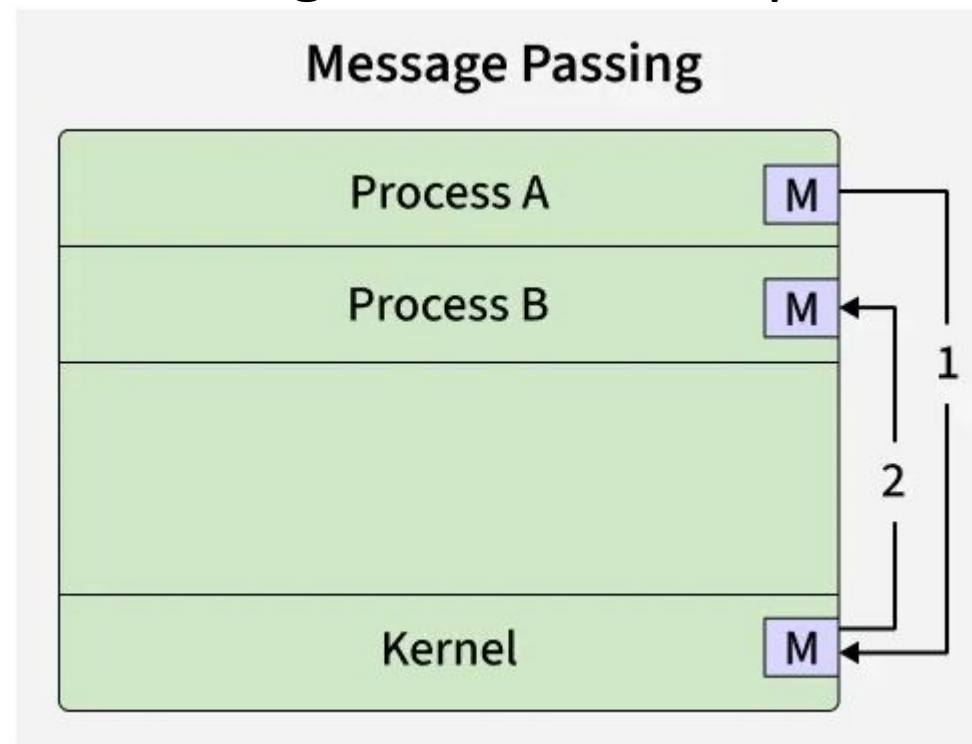
- Communication between processes using shared memory requires processes to share some variable and it completely depends on how the programmer will implement it. Processes can use shared memory for extracting information as a record from another process as well as for delivering any specific information to other processes.



- **Shared Memory** In the above shared memory model, a common memory space is allocated by the kernel.
- Process A writes data into the shared memory region (Step 1).
- Process B can then directly read this data from the same shared memory region (Step 2).
- Since both processes access the same memory segment, this method is fast but requires synchronization mechanisms (like semaphores) to avoid conflicts when multiple processes read/write simultaneously.
- Example: Multiple people can edit the document at the same time in shared google doc.

2. Message Passing

- Message Passing is a method where processes communicate by sending and receiving messages to exchange data. One process sends a message and the other process receives it, allowing them to share information. Message Passing can be achieved through different methods like Sockets, Message Queues or Pipes.



Message Passing

- In the above message passing model, processes exchange information by sending and receiving messages through the kernel.
- Process A sends a message to the kernel (Step 1).
- The kernel then delivers the message to Process B (Step 2).
- Here, processes do not share memory directly. Instead, communication happens via system calls (`send()`, `recv()`, or similar).
- This method is simpler and safer than shared memory because there's no risk of overwriting shared data, but it incurs more overhead due to kernel involvement.
- Example: Multiple people send updates to a group chat, but each message goes through the server before others see it, like processes sending messages instead of directly sharing memory.

Inter-process synchronization

- Inter-process synchronization coordinates concurrent processes or threads to ensure data consistency, prevent race conditions, and manage shared resources in computer architecture.
- It enforces ordered execution, often requiring mutual exclusion to ensure only one process accesses critical sections at a time.
- Common techniques include semaphores, mutexes, barriers and spinlocks.

1. Mutex (Mutual Exclusion)

- **Definition:** A locking mechanism that ensures only one thread accesses a resource at a time.
- **Analogy: A Single-Occupancy Bathroom Key.** Only the person holding the key can enter. If someone else wants to use it, they must wait in line (sleep) until the occupant leaves and returns the key.
- **Key Property:** The same thread that locks it *must* unlock

2. Semaphores

- **Definition:** A counter that restricts access to a limited number of resources.
- **Analogy: A Library with 10 Study Rooms.** The front desk has 10 keys.
 - **Counting:** If all keys are gone, the next student waits (sleeps).
 - **Signaling:** Any librarian (even one who didn't take a key) can take a returned key and give it to the next student.
- **Key Property:** Can be binary (0 or 1) or counting

3. Spinlocks

- **Definition:** A lock where the thread continuously polls (spins) while waiting for the lock to become available, rather than sleeping.
- **Analogy: A Small, Popular Espresso Machine.** You stand directly in front of the machine, repeatedly asking "Is it done yet?" every second until the previous user leaves. It is efficient if the wait is short, but wastes CPU power if long.
- **Key Property:** Best for very short, non-blocking critical sections; rarely used in user space.

4. Barriers

- **Definition:** A point where all participating threads must stop and wait until all other threads have reached the same point, at which point they are all released.
- **Analogy: A Hiking Group at a Trailhead.** The group agrees that no one starts hiking until *everyone* has arrived at the parking lot.
- **Key Property:** Synchronizes execution flow, not just resource access
- continue.

Summary Comparison Table

Feature 	Mutex	Semaphore	Spinlock	Barrier
Mechanism	Locking (Exclusive)	Signaling (Counting)	Busy-Waiting	Instruction Ordering
Waiting	Sleep-Wait	Sleep-Wait	Busy-Wait	No Blocking
Ownership	Yes (Acquirer must Release)	No	Yes (Usually)	N/A
Use Case	Mutual Exclusion	Resource Pools	Short, High-Speed Waits	Memory Consistency
Overhead	High (Context Switch)	High (Context Switch)	Low (No Switch)	Minimal

Cache Coherence

- Cache coherence is the problem of ensuring that multiple processors (brains of a computer) working on the same data see the same, up-to-date information.
- Cache coherence is the consistency and synchronization of shared data stored in multiple private caches within a multicore or multiprocessor system. It ensures that if one processor updates a memory location, all other processors see that updated value rather than stale data. It is vital for maintaining data integrity in parallel computing

Key Aspects of Cache Coherence:

The Problem: When multiple cores have independent caches (L1/L2), they may hold different values for the same memory address. If core A modifies a variable, core B might still hold the old value.

Coherence Protocols: These are mechanisms that manage cache states and enforce consistency. They manage data movement and invalidation to ensure a uniform view of memory.

Solution of Cache coherence:

1. The "Snooping" Solution (Everyone Watches)

In this approach, all caches are connected to a shared bus (like a central loudspeaker system).

- How it works: When one processor changes data in its cache, it "shouts" this change over the bus. Every other cache "snoops" (listens) to the bus.
- The Action: If another processor hears that data it holds has been changed, it immediately marks its own copy as invalid (outdated).
- Analogy: If Chef A yells, "I added salt to the sauce!" Chef B instantly throws away their old sauce copy and knows to get the new one

2. The "Directory" Solution (The Librarian)

- For complex systems with many processors, "snooping" is too noisy. Instead, a central "Directory" (a librarian) keeps track of who has which piece of data.
- How it works: Before a processor can change data, it must ask the directory for permission.
- The Action: The directory tells all other processors holding that data to invalidate (delete) their copies before the change happens.
- Analogy: A head chef (Directory) tracks who has which ingredient. If Chef A wants to change the sugar, they ask the head chef, who tells all other chefs to stop using the old sugar

3. The "Write-Through" Solution (Instant Update)

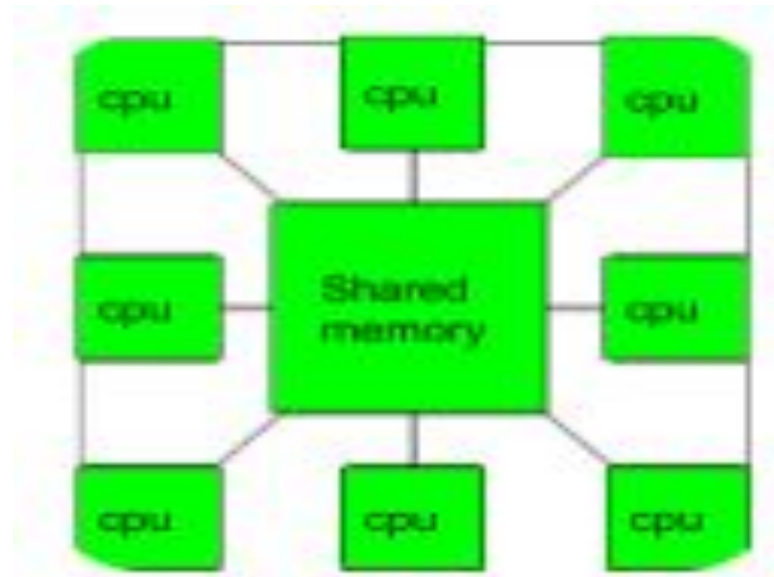
- This approach ensures the main memory is always up-to-date.
- How it works: Whenever a processor writes (changes) data in its cache, it immediately writes that same change to the main memory as well.
- The Action: Since the main memory is always accurate, other processors can just check it if their own cache copy is outdated.
- Analogy: Every time a chef adds salt, they instantly send a report to the main pantry.

Summary Table of Solutions

Solution 	How it works	Analogy
Snooping	Caches "listen" to a bus for updates.	Chefs listening to someone yelling updates.
Directory	A manager keeps track of who has what.	A librarian directing information flow.
Write-Through	Changes are instantly updated in main memory.	Instant, real-time reporting of changes.

Shared memory multiprocessors

- A shared memory multiprocessor is a computer architecture where multiple processors or cores share a single common physical address space, allowing them to communicate and access data directly. These systems typically use cache coherence protocols to ensure data consistency, facilitating efficient parallel computing for tasks requiring high performance.



Key Characteristics and Types:

- Symmetric Multiprocessing (SMP): All processors are identical and access memory with similar latency (Uniform Memory Access - UMA).
- Non-Uniform Memory Access (NUMA): Processors are grouped, and memory access latency depends on the physical location of the memory relative to the processor (Distributed Shared Memory - DSM).
- Cache Coherence: Protocols such as MESI (Modified, Exclusive, Shared, Invalid) are crucial to ensure that when one processor modifies a memory location, all other processors see the updated value.

Advantages and Limitations

- **Advantages:** Simplified communication between processes and easy sharing of data.
- **Limitations:** Limited scalability due to memory contention and high bandwidth requirements, especially with a single shared bus

*****End*****

